MITCON-CREDENTIA

Project Handover Documentation

Field	Value
Date	2026-07-27
Prepared by	Vibin Cariappa (Data Analyst Intern, BDS)
Status	Active Production
Platforms	Frontend → Vercel | Backend → Railway | Database → Supabase

1. Project Overview

MITCON Credentia is a Secure Document Repository and Ledger System built for MITCON Credentia, Bengaluru. It tracks physical legal and financial documents (e.g., Debenture Trust Deeds, Promissory Notes, Share Pledge Agreements) held in the MITCON vault for various clients.

Core Capabilities

Feature	Description
Vault Repository	Register, view, search, and delete physical documents
Document Checkout	Employees can check out documents with digital signature capture
Document Return	Full return workflow with condition assessment and re-signature
Notification System	Real-time push notifications via Socket.IO + browser desktop notifications
User & Role Management	RBAC with three roles: super-admin, admin, developer
Security Policy	Configurable global security policies (session timeout, MFA, upload formats)
Compliance Reports	CSV/print export of checkout history and document status
Backup & Restore	Full JSON backup/restore of all database tables
Seed Restore	Restore initial document library via /api/documents/restore-seed

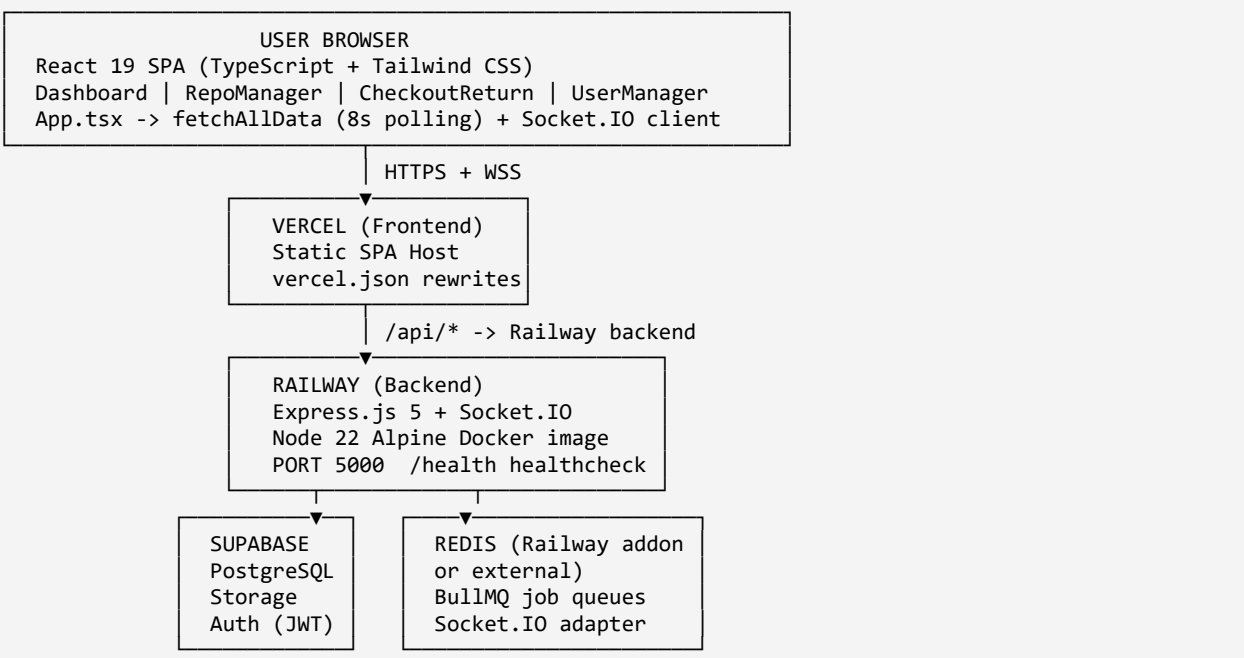
2. Repository Structure

Top-level monorepo layout (backend = Express/Node, frontend = React 19 + Vite):

```
MITCON-CREDENTIA/                  <- Monorepo root
├── .env, .gitignore, .npmrc
├── package.json                   <- Root workspace package
├── prisma.config.ts               <- Root Prisma config pointer
├── railway.json                   <- Railway root deploy config (Nixpacks)
├── brain.md / hld.md               <- Internal notes / High-Level Design
│
├── backend/                       <- Express.js Node.js backend
```

```
│   ├── Dockerfile, docker-compose*.yml, docker-entrypoint.sh
│   ├── railway.json                <- Railway backend config (Dockerfile builder)
│   ├── nginx/nginx.conf
│   ├── scripts/                     (diagnose-supabase.js, healthcheck.sh, migrate.sh, seed.sh,
start.sh)
│   ├── prisma/  (schema.prisma, seed.js, migrations/)
│   └── src/
│       ├── server.js, app.js, data-store.json (legacy)
│       ├── auth/auth.routes.js
│       ├── routes/  (documents, checkout, security, backup, notification)
│       ├── controllers/ , services/ , repositories/   (future-ready, partially wired)
│       ├── config/  (env, database, supabase, redis, bullmq, socket, cors, helmet, logger, ...)
│       ├── middleware/  (auth, upload, rateLimit, error, socket, security, ...)
│       ├── jobs/  (index.js, report.job.js)
│       ├── utils/ , validations/ , constants/ , shared/
│
└── frontend/                       <- React 19 + Vite + TypeScript + Tailwind
    ├── vercel.json, vite.config.js, tailwind.config.js, index.html
    ├── Dockerfile, nginx.conf, PRODUCTION_READINESS.md
    ├── api/       <- Vercel serverless API proxy function
    ├── cypress/   <- E2E test specs (empty, configured)
    └── src/
        ├── main.tsx, App.tsx, types.ts, index.css
        └── components/  (LoginPage, Dashboard, RepoManager, CheckoutReturn,
                          UserManager, ReportModule, NotificationCenter, SignatureCanvas)
```

3. Architecture Diagram



4. Deployment Topology

4.1 Frontend — Vercel

Property	Value
Platform	Vercel
Framework	Vite (React SPA)
Build command	npm run build (from frontend/)
Output directory	frontend/dist
Root directory	frontend/
Config file	frontend/vercel.json
Node.js version	22.x

vercel.json rewrites /api/:path* to /api/proxy — an internal serverless function that forwards requests to the Railway backend. The Vercel environment variable VITE_API_URL must point to the Railway backend URL.

Frontend environment variables (set in Vercel dashboard)

```
VITE_APP_NAME=MITCON Credentia Portal
VITE_API_URL=https://<your-railway-backend-url>
VITE_SOCKET_URL=https://<your-railway-backend-url>
VITE_SUPABASE_URL=https://<your-supabase-project>.supabase.co
VITE_SUPABASE_ANON_KEY=<supabase_anon_key>
```

4.2 Backend — Railway

Property	Value
Platform	Railway
Builder	Dockerfile (backend/Dockerfile)
Config file	backend/railway.json
Health check	GET /health (30s timeout)
Restart policy	ON_FAILURE
Exposed port	5000
Start command	npm start → node src/server.js
Entrypoint	docker-entrypoint.sh (runs Prisma migrate deploy first)

Backend environment variables (set in Railway dashboard → Variables)

```
NODE_ENV=production
PORT=5000
DATABASE_URL=<supabase_postgres_pooler_url>
DIRECT_URL=<supabase_postgres_direct_url>
REDIS_URL=<railway_redis_url_or_external>
REDIS_ENABLED=true
SUPABASE_URL=https://<your-supabase-project>.supabase.co
SUPABASE_ANON_KEY=<supabase_anon_key>
SUPABASE_SERVICE_ROLE_KEY=<supabase_service_role_key>
SUPABASE_JWT_SECRET=<supabase_jwt_secret>
CORS_ORIGINS=https://<your-vercel-app>.vercel.app
JWT_SECRET=<random_min_32_chars>
JWT_EXPIRY=1h
```

```
BCRYPT_ROUNDS=12
LOG_LEVEL=info
```

CORS caution

CORS_ORIGINS must include the exact Vercel deployment URL. The CORS config in backend/src/config/cors.js also auto-allows any *.vercel.app origin containing the string "mitcon-credentia" — a convenience shortcut that should be reviewed before strict production use.

4.3 Database — Supabase

Property	Value
Platform	Supabase (PostgreSQL)
ORM	Prisma 5.x
Schema file	backend/prisma/schema.prisma
Migration history	backend/prisma/migrations/
Seeder	backend/prisma/seed.js

Two connection strings are required: DATABASE_URL (pooler, port 6543, used at runtime) and DIRECT_URL (direct, port 5432, used by migrations). Find these at Supabase Dashboard → Project Settings → Database → Connection string.

Storage buckets (create manually in Supabase Storage before first deploy)

Bucket	Purpose	Max Size	Access
mc-documents	Physical document files	50 MB	Private
mc-signatures	Digital signature images	5 MB	Private
mc-reports	Generated PDF/Excel reports	20 MB	Private (expires 30 days)
mc-temporary	Temp upload staging	100 MB	Private (expires 24h)

5. Frontend — Deep Dive

Technology Stack

Library	Version	Purpose
React	19.x	UI framework
TypeScript	~5.7	Type safety
Vite	8.x	Build tool + dev server
Tailwind CSS	3.x	Utility CSS styling
React Router DOM	7.x	Client-side routing
Socket.IO Client	4.x	Real-time WebSocket
Zustand	5.x	Global state management

Library	Version	Purpose
TanStack React Query	5.x	Server state + caching
React Hook Form	7.x	Forms
Zod	4.x	Schema validation
Axios	1.x	HTTP client (configured, not primary — fetch is used)
Recharts	3.x	Data visualization
Lucide React	1.x	Icon library
react-toastify	11.x	Toast notifications

Application State (App.tsx)

The root App.tsx owns all server-synced state. It does not use React Query for data fetching — instead it uses a custom fetchAllData() function with Promise.all, called on initial mount, every 8 seconds (polling), and on every Socket.IO notification:new event.

LocalStorage keys used: bcd_user (serialized logged-in user object), bcd_token (auth token string), bcd_active_tab (last active navigation tab).

Pages / Components

Component	Route Tab	Description
LoginPage.tsx	(pre-auth)	Email + password form, POST /api/auth/login
Dashboard.tsx	dashboard	KPI cards: total docs, active checkouts, returned, users. Desktop notification prompt.
RepoManager.tsx	repo	Full document vault: search, filter by client, add new doc, soft-delete, navigate to checkout
CheckoutReturn.tsx	checkouts	Tabbed: Active Checkouts list, Return workflow with condition form + signature
UserManager.tsx	users	User CRUD (email domain restricted) + Security Policy editor
ReportModule.tsx	reports	CSV export of checkouts, returns, document audit trails + print capability
NotificationCenter.tsx	(header)	Bell icon + dropdown of recent notifications, mark-read, clear-all
SignatureCanvas.tsx	(embedded)	3-mode canvas: draw with mouse/touch, type name (hash), upload image

The navigation bar currently shows all tabs regardless of role. RBAC tab filtering is not enforced at the navigation level — only backend endpoints enforce it. Role-based tab visibility is a recommended follow-up.

Socket.IO Client Connection (App.tsx)

```
const socketUrl = import.meta.env.VITE_SOCKET_URL || "http://localhost:5000";
const socket = io(socketUrl, { auth: { token } });
```

Vite Dev Proxy

In development, Vite proxies /api/* and /socket.io/* to http://127.0.0.1:5000 (see vite.config.js).

Build Chunk Splitting

Production builds split vendor chunks: vendor-react, vendor-state, vendor-forms, vendor-charts, vendor-supabase, vendor-other.

Fonts

Loaded from Google Fonts in index.html: Inter (400–800, primary UI) and IBM Plex Sans (400–500, secondary).

6. Backend — Deep Dive

Technology Stack

Library	Version	Purpose
Express.js	5.x	HTTP framework
Prisma	5.x	ORM + migrations
Socket.IO	4.x	WebSocket server
BullMQ	5.x	Background job queues
ioredis	5.x	Redis client
@supabase/supabase-js	2.x	Storage + Auth SDK
bcrypt	5.x	Password hashing
jsonwebtoken	9.x	JWT utilities
multer	2.x	File upload handling
pdfkit	0.19.x	PDF generation
exceljs	4.x	Excel report generation
pino	9.x	Structured logging
zod	3.x	Env schema validation
helmet	7.x	HTTP security headers
cors	2.x	CORS middleware
compression	1.x	Gzip response compression
express-rate-limit	7.x	Rate limiting
cookie-parser	1.x	Cookie parsing

Middleware Chain (in order)

- performanceMiddleware — tracks request durations
- corsMiddleware — CORS whitelist enforcement
- helmetMiddleware — security HTTP headers
- requestIdMiddleware — injects X-Request-Id correlation ID
- requestLogger — Pino HTTP request logger
- cookieParserMiddleware — parses Cookie headers
- jsonParser — express.json() with size limits
- urlEncodedParser — express.urlencoded() with size limits
- compressionMiddleware — Gzip compression
- apiLimiter — 10,000 requests per 15 minutes per IP

Post-routing: errorLogger intercepts and logs errors, then errorMiddleware formats all errors into a standard JSON response.

Route Architecture

Active routes query Prisma directly (not through controllers/services). The controllers/, services/, and repositories/ directories exist as a future-ready architecture but are not yet wired into the main routes — the simpler direct-Prisma approach is what is deployed.

```
POST   /api/auth/login              (no auth required)
GET    /api/documents
POST   /api/documents
DELETE /api/documents/:id
POST   /api/documents/restore-seed  restore initial seed documents
GET    /api/checkouts
POST   /api/checkouts               create checkout (updates document status)
POST   /api/checkouts/:id/return    process document return
GET    /api/returns
GET    /api/users
POST   /api/users                   create user (domain-restricted)
DELETE /api/users/:id
GET    /api/policies
PUT    /api/policies
GET    /api/backup                  export full database as JSON
POST   /api/backup/restore          restore database from JSON backup
GET    /api/notifications
PUT    /api/notifications/:id/read
POST   /api/notifications/clear-all
GET    /health                      health check probe (no auth)
```

Authentication — Current State

> **Not production-ready**
> The current authentication implementation is a mock/prototype and must not be used as-is in production.

How it works today:

- User submits email + password to POST /api/auth/login.
- Backend looks up the user by email in the users table.
- The submitted password is checked against a small hardcoded list of accepted passwords rather than a securely hashed value.

- On success, the server issues a non-cryptographic placeholder token that simply encodes the user's email.
- The auth middleware decodes that token by parsing the email back out of it and loading the user record — it does not verify a signature.
- Practical impact: because the token is just the user's email in a fixed wrapper, anyone who knows (or guesses) a valid user email can construct a token that grants that user's access. This must be replaced with properly signed JWTs and hashed passwords before the system holds sensitive data.

What needs to be implemented for real auth

- Replace the placeholder password check with bcrypt-hashed passwords stored in Supabase Auth or the users table.
- Sign proper JWTs using jsonwebtoken with JWT_SECRET.
- Implement a token refresh flow.
- Consider using Supabase Auth (already a dependency) for OAuth/email login.

Server Bootstrap Sequence

- Load .env via dotenv.
- Validate env schema via Zod (exits with code 1 if invalid).
- Connect to PostgreSQL via Prisma (SELECT 1 check).
- Ping Redis connection.
- List Supabase storage buckets (connectivity check).
- Initialize Socket.IO server.
- Start HTTP server on PORT (default 5000).
- In production, DB / Redis / Storage failures all exit the process. In development, only a DB failure is blocking.

Graceful Shutdown (SIGTERM / SIGINT)

- Close HTTP server (stop accepting connections).
- Close Socket.IO server.
- Shutdown BullMQ queues and workers.
- Quit Redis connection.
- Exit with code 0 — with a 10-second force-quit safety timeout.

Logging

Uses Pino: pino-pretty (colorized) in development, JSON structured logs in production (collected by Railway). Redacts Authorization headers and password/token fields.

7. Database — Prisma & Supabase

User (table: users)

Field	Type	Notes
id	String PK	Employee-style identifier
name	String	
email	String, unique	Restricted to approved company email domains
role	String	super-admin, admin, developer
status	String	default active; also suspended
designation	String?	Optional job title
createdAt	DateTime	

Document (table: documents)

Field	Type	Notes
id	String PK	e.g. doc-1, doc-1234567890
documentId	String, unique	UUID (auto-assigned on create)
documentName	String	e.g. "DTD", "Promissory Note"
dateUploaded	DateTime	
expiryDate	String?	Optional
filePath	String	secure/repository/<id>.pdf — path in Supabase Storage
status	String	Available, Checked Out
uploadedBy	String	User name or "System"
client	String	Client company name
dateOfRegistration	String	Date string
placeOfHolding	String	default "Bengaluru Office"

Checkout (table: checkouts)

Field	Type	Notes
id	String PK	chk-<timestamp>
documentId	String	UUID tracking ID from Document
documentDbId	String	DB primary key of Document
documentName	String	
employeeName	String	

Field	Type	Notes
employeeId	String?	
designation	String?	
checkoutDate	String	ISO date string
destination	String	
purpose	String	
expectedReturnDate	String	
approvalAuthority	String	default "Self Check"
status	String	Checked Out, Returned
signature	String?	Base64 image data or typed text
signatureType	String?	drawn, uploaded, typed

Return (table: returns)

Field	Type	Notes
id	String PK	ret-<timestamp>
checkoutId	String	FK to Checkout.id
documentId	String	
documentName	String	
returnDate	String	ISO date
returnTime	String	locale time string
condition	String	Perfect, Good, Damaged, etc.
notes	String?	
returningEmployeeSignature	String?	Base64
returningEmployeeName	String	

Notification (table: notifications)

Field	Type	Notes
id	String PK	not-<timestamp>
title	String	
message	String	
status	String	unread, read
timestamp	DateTime	

SecurityPolicy (table: security_policies)

Field	Type	Default
key	String PK	"global_policy" (singleton)
passwordMinLength	Int	8
requireMfa	Boolean	false
sessionTimeoutMinutes	Int	30
allowedUploadFormats	String[]	["pdf","docx","xlsx"]
autoRejectExpiredCheckouts	Boolean	false
maxCheckoutDurationDays	Int	30

Migration Commands

```
cd backend
npx prisma migrate dev --name <migration_name>   # create migration (dev)
npx prisma migrate deploy                        # apply pending migrations (prod)
npx prisma generate                               # regenerate client after schema change
npx prisma migrate reset                          # reset database (DESTRUCTIVE — dev only)
```

Seeding

node prisma/seed.js wipes all tables and re-seeds with: 4 core users (credentials distributed separately — see Section 18), all initial documents from initialDocuments.js (~100+ documents across multiple clients), a default security policy, and 1 welcome notification.

8. Environment Variables Reference

Backend (backend/.env)

Variable	Required	Default	Description
NODE_ENV	Yes	development	development, production, test
PORT	No	5000	HTTP server port
LOG_LEVEL	No	info	debug, info, warn, error
APP_NAME	No	MITCON Ledger	Application display name
APP_URL	No	http://localhost:5000	Backend base URL
DATABASE_URL	Yes	—	Supabase PostgreSQL pooler connection string
DIRECT_URL	No	(same as DATABASE_URL)	Supabase direct connection string
SUPABASE_URL	Yes	—	Supabase project URL
SUPABASE_SERVICE_ROLE_KEY	Yes	—	Service role key (bypasses RLS)

Variable	Required	Default	Description
SUPABASE_ANON_KEY	No	fallback	Public anon key
SUPABASE_JWT_SECRET	No	fallback	JWT secret for decoding
SUPABASE_BUCKET	No	documents	Default bucket name
JWT_SECRET	No	fallback string	JWT signing secret (min 8 chars)
JWT_EXPIRY	No	1h	Token TTL
REFRESH_SECRET	No	fallback string	Refresh token secret
SESSION_SECRET	No	fallback string	Session secret
REDIS_URL	No	redis://localhost:6379	Full Redis connection URL
REDIS_ENABLED	No	true	Set to false to disable Redis (mock mode)
REDIS_HOST	No	localhost	Redis hostname
REDIS_PORT	No	6379	Redis port
REDIS_PASSWORD	No	—	Redis password
CORS_ORIGIN	No	—	Single CORS origin override
CORS_ORIGINS	No	http://localhost:3000	Comma-separated CORS whitelist
RATE_LIMIT_WINDOW	No	15	Rate limit window in minutes
RATE_LIMIT_MAX	No	100	Max requests per window
SMTP_HOST	No	—	Email server host
SMTP_PORT	No	587	Email server port
SMTP_USER	No	—	SMTP username
SMTP_PASSWORD	No	—	SMTP password

Frontend (frontend/.env)

Variable	Required	Description
VITE_APP_NAME	No	Display name in browser
VITE_API_URL	Yes (prod)	Full Railway backend URL for API calls
VITE_SOCKET_URL	Yes (prod)	Full Railway backend URL for Socket.IO
VITE_SUPABASE_URL	No	Supabase URL (for direct client auth if used)
VITE_SUPABASE_ANON_KEY	No	Supabase public key

9. Real-Time System (Socket.IO)

Initialized on top of the HTTP server. CORS is read from CORS_ORIGIN (or * if unset). Ping timeout 60s, ping interval 25s. A socket auth middleware validates the Bearer token supplied in the handshake.

Socket Rooms

user:<userId> — per-user events

role:<role> — role-broadcast events

department:<departmentId> — department broadcast (if user.departmentId exists)

Events — Server → Client

Event	Payload	Trigger
notification:new	Notification object	Document created, checkout created, document returned
notification:count:update	{ unreadCount }	After mark-read
notification:updated	{ id, status }	After single mark-read
notification:read	{ allRead: true }	After mark-all-read

Events — Client → Server

Event	Payload
mark_notification_read	{ notificationId }
mark_all_notifications_read	(none)
notification_acknowledged	{ notificationId }

10. Background Job System (BullMQ + Redis)

Queue	Name	Concurrency	Purpose
Audit	audit-queue	10	Audit log DB insertions
Notification	notification-queue	5	Notification dispatching
Preview	preview-queue	2	File thumbnail generation (CPU-heavy)
Report	report-queue	1	Compliance report generation
Scheduler	scheduler-queue	3	Scheduled tasks
Virus Scan	virus-scan-queue	5	ClamAV file scanning

If REDIS_ENABLED=false (or Redis is unreachable), all queues/workers use mock implementations. The server boots and runs without Redis — only background jobs go offline. This is the development-friendly default.

Job Retry Configuration

- Attempts: 3
- Backoff: exponential, starting at 5 seconds
- Completed jobs kept: 24 hours or 1,000 records
- Failed jobs kept: 7 days or 5,000 records

Status of Worker Implementations

Most workers are placeholder skeletons and do not execute real business logic: processAuditJob, processNotificationJob, processPreviewJob, and processSchedulerJob only log. processReportJob is partially implemented (delegates to ReportService). processVirusJob only logs.

11. File Storage (Supabase Storage)

Two Supabase clients are exported from backend/src/config/supabase.js: a public anon client (safe for client-facing identity checks) and an admin client using the service-role key (bypasses all RLS — backend only).

Allowed Upload Formats by Bucket

Bucket	Extensions	MIME Types
mc-documents	pdf, docx, xlsx, pptx, jpg, png, zip	Standard office + image
mc-signatures	png, jpg, jpeg, pdf	Image types
mc-reports	pdf, xlsx, csv	Report formats
mc-temporary	Same as documents	Same as documents

Upload Validation Pipeline

- Filename sanity — rejects special characters
- Extension check — against bucket allowed list
- MIME type check — against bucket allowed MIME types
- File size check — against bucket max size
- Magic byte verification — validates actual file header bytes against declared MIME type (prevents disguised executables)

Not fully wired

Supabase Storage is configured in the backend but physical file uploads are not yet fully connected through the active routes — routes currently create document metadata records only. Actual uploads need to go through storage.service.js.

12. Security Layers

Layer	Implementation
HTTPS	Enforced by Vercel (frontend) and Railway (backend) — both provide TLS termination
CORS	Whitelist in cors.js — CORS_ORIGINS values plus a *.vercel.app pattern match
Helmet	HTTP security headers (CSP, HSTS, X-Frame-Options, etc.)

Layer	Implementation
Rate Limiting	10,000 req/15min global, 5 req/15min for auth, 20 req/hr for downloads
Magic Byte Validation	File upload verifies actual file header bytes
Request ID Tracing	Every request gets an X-Request-Id correlation ID
Error Masking	In production, non-operational errors return generic messages
Token Redaction	Pino logger redacts Authorization headers and password fields
Email Domain Lock	User creation restricted to approved company email domains
CSP Header	Strict Content-Security-Policy in index.html

RBAC not enforced
Most routes do not use the requireAuth middleware. The next developer must add authentication guards to all non-public endpoints.

13. Known Issues & Technical Debt

Caution
The following issues must be addressed before the system handles sensitive data in production.

Critical Issues

Issue	File	Impact
Mock authentication	auth.routes.js	Placeholder password check, no real JWT signing — tokens can be forged
No auth guards on routes	All route files	Most endpoints are publicly accessible without a valid token
approvalRequest model missing	backup.routes.js	References prisma.approvalRequest, which does not exist in schema.prisma — causes a runtime crash on /api/backup
owner field missing	backup.routes.js	Restore route sets owner on Document, which has no owner field in schema

Non-Critical Issues

Issue	File	Notes
Prisma $disconnect() commented out	server.js	Not disconnecting Prisma gracefully on shutdown
Services/controllers/repositories not wired	controllers/, services/, repositories/	Exist but unused by active routes; planned future architecture

Issue	File	Notes
data-store.json exists	src/data-store.json	Legacy JSON file from early development; no longer used
brain.md and hld.md at root	Root	Internal development notes; should not be in a production repo
CSP connect-src hardcodes Supabase URL	index.html	Update if the Supabase project changes
Frontend uses fetch, not Axios	App.tsx	Axios is installed but native fetch is used for all API calls
No persistent sessions	App.tsx	Session is lost on page reload if localStorage is cleared
husky install deprecation	frontend/package.json	prepare script uses deprecated husky install; harmless but noisy

14. Local Development Setup

Prerequisites

- Node.js 22.x
- npm 10.x
- PostgreSQL (via Supabase or local)
- Redis (optional — set REDIS_ENABLED=false to skip)

Steps

```
# 1. Clone the repository
git clone <repo-url> && cd MITCON-CREDENTIA

# 2. Backend
cd backend
cp .env.example .env        # fill in your own Supabase credentials
npm install
npx prisma generate
npx prisma migrate dev --name init
node prisma/seed.js         # seed the database
npm run dev                 # http://localhost:5000

# 3. Frontend (new terminal)
cd frontend
cp .env.example .env        # can be mostly empty for local dev
npm install
npm run dev                 # http://localhost:3000
```

Verify Setup

- Navigate to http://localhost:3000
- Log in using one of the seed accounts (credentials distributed separately — see Section 18)
- Dashboard should show loaded documents

- Backend health check: http://localhost:5000/health

15. Production Deployment Runbooks

Deploy Backend to Railway

- Push changes to the connected Git branch.
- Railway auto-detects changes in backend/ (per watchPatterns).
- Docker image is built using backend/Dockerfile.
- docker-entrypoint.sh runs prisma migrate deploy before starting the server.
- Health check hits GET /health — must return 200 within 30s.
- Manual redeploy: Railway dashboard → Project → Deploy → Redeploy. Logs: Deployments → View Logs.

Deploy Frontend to Vercel

- Push changes to the connected Git branch.
- Vercel auto-builds using npm run build in frontend/.
- Output dist/ is deployed as static files.
- vercel.json rewrites /api/* to the Vercel serverless proxy function in frontend/api/.
- Manual redeploy: Vercel dashboard → Project → Deployments → Redeploy.

Adding a New Database Migration

```
# 1. Modify schema.prisma
cd backend
npx prisma migrate dev --name add_your_feature
# 2. Push to Railway — docker-entrypoint.sh auto-runs `npx prisma migrate deploy`
```

Reseed Production Database

> **Destructive**
> This deletes all existing records.

```
# On Railway: open the Shell tab in your deployment
node prisma/seed.js

# Or via the API (only restores documents, not users/policy):
POST /api/documents/restore-seed
```

16. Seed Users

Name	Role	Employee ID
Ankita Agrawal	super-admin	10841
Ravi Injolkar	admin	90092
Mahesh Madhavarm	admin	11150

Note: the current backend also accepts a couple of hardcoded generic passwords for quick testing. These must be removed before production use — see Section 6, Authentication.

Role Capabilities

Role	Can Do
super-admin	Everything — bypasses all role checks
admin	All operations
developer	Limited (no explicit frontend tab locks currently)

17. File-by-File Inventory

Root Level

File	Purpose
.env	Root-level environment (used when running Prisma from root)
.gitignore	Ignores node_modules/, .env*, coverage/, logs/, .DS_Store
.npmrc	legacy-peer-deps=true to handle dependency conflicts
package.json	Root workspace entry; holds root-level prisma config
prisma.config.ts	Points Prisma to backend/prisma/schema.prisma
railway.json	Alternative Nixpacks-based Railway config (root deploy config)
brain.md	AI agent development log — not for production
hld.md	High-level design document — architecture planning artifact

Backend Files

File	Purpose
src/server.js	HTTP server bootstrap, graceful shutdown, health checks
src/app.js	Express app factory — middleware chain + all route mounts
src/auth/auth.routes.js	Login endpoint (mock auth — see Section 6)
src/routes/documents.routes.js	Document CRUD + seed restore
src/routes/checkout.routes.js	Checkout + return flows
src/routes/security.routes.js	Users + policies
src/routes/backup.routes.js	Full DB backup/restore (has bug — see Known Issues)
src/routes/notification.routes.js	Notification read/clear
src/config/env.js	Zod schema validation of all env vars, frozen config object
src/config/database.js	Prisma singleton with slow query monitoring

File	Purpose
src/config/supabase.js	Supabase anon + admin clients + storage bucket configs
src/config/redis.js	ioredis client factory + mock client fallback
src/config/bullmq.js	BullMQ connection options + queue configs
src/config/socket.js	Socket.IO server init + room management + event handlers
src/config/cors.js	CORS whitelist from CORS_ORIGINS env var
src/config/helmet.js	Helmet HTTP security headers config
src/config/logger.js	Pino logger (pretty in dev, JSON in prod)
src/config/initialDocuments.js	Static array of ~100+ real client documents for seeding
src/jobs/index.js	BullMQ queue + worker instantiation (6 queues, 6 workers)
src/jobs/report.job.js	Report job processor helper
src/middleware/auth.middleware.js	requireAuth / requireRole / requirePermission
src/middleware/upload.middleware.js	Multer + magic byte validation
src/middleware/rateLimit.middleware.js	API/auth/download rate limiters
src/middleware/error.middleware.js	Global error formatter
src/middleware/socket.middleware.js	Socket.IO auth middleware
src/middleware/compression.middleware.js	Gzip response compression
src/middleware/performance.middleware.js	Request duration tracking
src/middleware/security.middleware.js	Additional security checks
src/middleware/validation.middleware.js	Zod request validation helper
src/middleware/asyncHandler.js	Wraps async route handlers to catch errors
src/middleware/errorLogger.js	Logs errors before error middleware
src/middleware/requestLogger.js	HTTP request logging via pino-http
src/shared/event-bus.js	Node.js EventEmitter singleton
src/shared/request-id.js	X-Request-Id header injector
src/utils/AppError.js	Custom operational error class
src/utils/error.util.js	Error normalization utility
src/utils/storage.util.js	File extension/MIME/size validation helpers
src/utils/notification.util.js	Notification creation helpers
src/utils/socket.util.js	Socket connection registry
src/utils/query.util.js	Prisma query helpers
src/utils/export.util.js	Export formatting utilities
src/utils/checkout.util.js	Checkout business logic helpers

File	Purpose
src/utils/documents.util.js	Document helpers
src/utils/security.util.js	Security validation helpers
src/utils/sanitizer.util.js	Input sanitization
src/utils/performance.util.js	Slow query detection
src/utils/logger.util.js	Logger wrapper helpers
src/constants/error.constants.js	Centralized error code constants
src/validations/checkout.validation.js	Zod schema for checkout requests
src/validations/documents.validation.js	Zod schema for document requests
prisma/schema.prisma	Database schema (6 models)
prisma/seed.js	Database seeder
scripts/diagnose-supabase.js	CLI tool to diagnose Supabase connectivity
scripts/healthcheck.sh	Shell health probe
scripts/migrate.sh / seed.sh / start.sh	Shell runners
docker-entrypoint.sh	Container startup: migrate then start server
Dockerfile	Multi-stage: builder (node:22-alpine) + runner (non-root nodejs user)
nginx/nginx.conf	Nginx reverse proxy config (used in Docker Compose deployments)

Frontend Files

File	Purpose
index.html	Entry HTML: CSP, Google Fonts, page title
src/main.tsx	React DOM render root
src/App.tsx	Root: auth state, data polling, Socket.IO, navigation
src/types.ts	TypeScript interfaces: User, Document, Checkout, Return, Notification, SecurityPolicy
src/index.css	Global base CSS
src/components/LoginPage.tsx	Login form with gradient background
src/components/Dashboard.tsx	KPI cards, active checkouts list, notification permission prompt
src/components/RepoManager.tsx	Document vault: search, filter, add, delete, checkout action
src/components/CheckoutReturn.tsx	Checkout request form + return processing form with signature
src/components/UserManager.tsx	User CRUD + security policy form
src/components/ReportModule.tsx	CSV download reports + print views
src/components/NotificationCenter.tsx	Bell icon + dropdown notification list
src/components/SignatureCanvas.tsx	Draw/type/upload signature component

File	Purpose
vercel.json	Vercel SPA fallback + API proxy rewrite
vite.config.js	Build config: chunk splitting, proxy, gzip
tailwind.config.js	Tailwind theme extension
nginx.conf	Nginx config for Docker-hosted frontend
Dockerfile	Frontend Docker image: build + nginx serve
PRODUCTION_READINESS.md	Deployment quality checklist
cypress.config.js	Cypress E2E config

18. Next Steps & Recommendations

Immediate (Before Any New Feature Work)

- Fix the backup route crash — add an ApprovalRequest model to schema.prisma or remove the approvalRequest references from backup.routes.js.
- Replace mock auth — implement real bcrypt password hashing and JWT signing; consider Supabase Auth for email/password login.
- Add requireAuth to all API routes — currently most routes are publicly accessible.
- Add VITE_SOCKET_URL to Vercel environment variables — without it, Socket.IO will try to connect to localhost in production.
- Create Supabase Storage buckets — mc-documents, mc-signatures, mc-reports, mc-temporary must be manually created.
- Rotate and securely distribute seed-user credentials — do not keep them in a shared document.

Short-Term Improvements

- Wire file uploads through Supabase Storage — currently documents are registered in the DB but no actual file is uploaded.
- Remove hardcoded/testing passwords from auth.routes.js.
- Remove data-store.json — legacy artifact no longer used.
- Remove brain.md and hld.md from the repository (or move to a separate docs repo).
- Wire controllers/services/repositories into the route layer — or simplify by removing the unused layers.

Architecture Improvements

- Add pagination to all list endpoints — currently returns all records, which will degrade with large datasets.
- Implement real RBAC — frontend tab access should be role-gated; backend routes should enforce roles.
- Connect BullMQ workers — implement audit logging, email notifications, and virus scanning.

- Replace 8-second polling with proper Socket.IO event broadcasts for all data changes.
- Add an audit log model to the Prisma schema for a compliance trail.

This document was generated from a review of the MITCON-CREDENTIA repository on 2026-07-27.